PROJECT REPORT

TITLE: AZURE AUTOMATION RUNBOOKS FOR VM START AND STOP SCHEDULES

Project Code: 24CC3046-P022

Department: Computer Science and Engineering (CSE)

College: KL University

Academic Year: 2026–2027

TEAM MEMBERS

Student ID	Student Name
2400031581	LAVETI JYOTHI SRI LAKSHMI DURGA SAMBHAVI
2400030511	RACHURI RAVI SHANKAR
2400030109	SAMUDRALA LOKESH
2400030090	KOPPU RUTHVIK

Team – T226

ABSTRACT

Azure Automation Runbooks provide a practical way to automate the starting and stopping of Azure Virtual Machines (VMs) according to predefined schedules. This project focuses on using Azure Automation to control VM power states automatically, particularly for non-production and test environments.

The proposed solution is intended to shut down non-production VMs outside business hours and reduce unnecessary cloud expenditure. A major expected benefit is the potential to cut the test-environment bill by approximately half by avoiding VM runtime when the environment is not required.

The project also considers two important operational and security bottlenecks: runbook failures can go unnoticed without appropriate alerting, and managed identity permissions may become over-scoped when permissions are granted for convenience. The design therefore emphasizes monitoring, alerting, and least-privilege access.

1. INTRODUCTION

Cloud platforms such as Microsoft Azure provide virtual machines (VMs) for development and testing. However, if these VMs are left running when they are not needed, they can increase cloud costs.

Our project focuses on using **Azure Automation Runbooks** to automatically start and stop VMs according to a fixed schedule. For example, a VM can be started in the morning and stopped after working hours.

The main goal is to reduce unnecessary VM usage, save cloud costs, and reduce manual work.

---

2. PROBLEM STATEMENT

Development and testing VMs do not always need to run throughout the day. If they are left running after working hours, they can increase the Azure bill.

Manually starting and stopping VMs every day is also inconvenient, and users may forget to do it.

Therefore, we propose an automated solution that can start and stop Azure VMs based on predefined schedules.

---

3. OBJECTIVES

The main objectives of this project are:

- Automatically start VMs during working hours.
- Automatically stop VMs outside working hours.
- Reduce unnecessary VM running time.
- Reduce cloud costs.
- Reduce manual work.
- Use Managed Identity for secure access.
- Monitor Runbook failures.
- Use only the required permissions.

---

4. USE CASES

4.1 Shutdown of Non-Production VMs

Development and testing VMs can be automatically stopped after working hours.

4.2 Automatic VM Startup

The required VMs can be automatically started at the beginning of the working day.

4.3 Cost Reduction

Stopping VMs when they are not required can reduce their running hours and help reduce cloud costs.

4.4 Reduced Manual Work

Users do not need to manually start and stop VMs every day.

---

5. REQUIREMENTS

5.1 Software Requirements

- Microsoft Azure Account
- Azure Portal
- Azure Virtual Machine
- Azure Automation Account
- Azure Automation Runbook
- Azure Managed Identity
- Azure Role-Based Access Control (RBAC)
- PowerShell
- Web Browser
- Microsoft Word

5.2 Functional Requirements

The proposed system should:

- Start selected VMs automatically.
- Stop selected VMs automatically.
- Run automation according to a schedule.
- Monitor Runbook execution.
- Identify Runbook failures.
- Provide alerts when automation fails.

5.3 Non-Functional Requirements

The system should be:

- **Secure** – Use Managed Identity and proper permissions.

- **Reliable** – Scheduled operations should work correctly.
- **Scalable** – Should support multiple VMs.
- **Cost-effective** – Should reduce unnecessary VM running time.
- **Easy to maintain** – Runbooks and schedules should be easy to update.

---

6. SDLC PLAN

The project will follow these main Software Development Life Cycle (SDLC) steps:

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

---

7. REQUIREMENT ANALYSIS

In this phase, we identify the problem and understand what the system needs to do.

The main requirement is to automatically start VMs during working hours and stop them after working hours.

Other requirements include:

- Scheduled VM operations.

- Secure authentication.
- Proper access permissions.
- Runbook monitoring.
- Failure alerts.
- Cost reduction.

---

8. SYSTEM DESIGN

The proposed system will use the following Azure components:

Azure VM → Automation Account → Runbook → Schedule

Main Components

Azure Virtual Machine:
The VM that needs to be automatically started or stopped.

Azure Automation Account:
Used to create and manage automation Runbooks.

Runbook:
Contains the automation script used to start or stop the VM.

Schedule:
Defines when the Runbook should execute.

Managed Identity:
Provides secure authentication without storing passwords in the automation script.

RBAC:
Controls the permissions given to the Managed Identity.

Monitoring and Alerts:
Used to identify Runbook failures.

9. PROPOSED IMPLEMENTATION

The project is planned to be implemented using the following steps:

1. Create an Azure Virtual Machine.
2. Create an Azure Automation Account.
3. Enable Managed Identity.
4. Assign the required RBAC permissions.
5. Create a Runbook for starting the VM.
6. Create a Runbook for stopping the VM.
7. Create schedules for the Runbooks.
8. Test the Runbooks.
9. Enable the schedules.
10. Configure monitoring and alerts.

Example schedule:

Operation	Example Time
Start VM	9:00 AM
Stop VM	6:00 PM

The actual timings can be changed according to the requirements.

---

10. TESTING PLAN

After implementation, the following test cases will be performed:

Test Case	Expected Result
Start Runbook	VM should start successfully
Stop Runbook	VM should stop successfully
Start Schedule	VM should start at the scheduled time
Stop Schedule	VM should stop at the scheduled time
Permission issue	Operation should fail safely
Runbook failure	Failure should be detected and alerted

Security testing will also be performed to make sure that the Managed Identity has only the required permissions.

---

11. EXPECTED RESULTS

At the end of the project, we expect the system to:

- Automatically start VMs during working hours.
- Automatically stop VMs after working hours.
- Reduce manual VM management.
- Reduce unnecessary VM running time.
- Help reduce cloud costs.
- Provide secure access using Managed Identity.
- Detect Runbook failures.

---

12. POSSIBLE BOTTLENECKS

Runbook Failures

A Runbook may fail because of permission problems, incorrect VM information, or other Azure issues.

Solution: Use monitoring and alerts to identify failed Runbook executions.

Over-Scoped Permissions

Giving the Managed Identity unnecessary permissions can create a security risk.

Solution: Follow the **principle of least privilege** and provide only the permissions required.

Incorrect Scheduling

An incorrect schedule could start or stop a VM at the wrong time.

Solution: Test the schedules carefully before regular use.

---

13. FUTURE ENHANCEMENTS

The project can be improved in the future by adding:

- Email or notification alerts.
- Different schedules for different environments.
- Tag-based VM selection.
- Cost monitoring dashboards.
- Support for multiple Azure subscriptions.
- Integration with Azure Monitor and DevOps tools.

---

14. CONCLUSION

The **Azure Automation Runbooks for VM Start and Stop Schedules** project aims to automate the management of Azure Virtual Machines.

The proposed solution will start VMs when they are needed and stop them when they are not required. This can reduce manual work, unnecessary resource usage, and cloud costs.

The project will also focus on security by using **Managed Identity and RBAC**, along with monitoring and alerts to identify automation failures.

Overall, this project will help us understand how Azure automation can be used to manage cloud resources efficiently.